

AGI ECOSYSTEM MASTER SPEC v2.0 — ENGINEERING PACK

1. SYSTEM ARCHITECTURE (PRODUCTION VIEW)

Layered system composed of: - Agent OS Runtime (execution kernel) - DAG Compiler (planning + dependency resolution) - Mythos Governance Engine (policy enforcement) - Event Store (immutable logs) - Swarm Runtime (distributed execution nodes) - Civilization Orchestrator (long-horizon coordination) - Simulation Engine (counterfactual evaluation)

2. DAG SPECIFICATION (FORMAL MODEL)

DAG = (Nodes, Edges) Node Types: - compute - memory_read - memory_write - validate - agent_task Constraints: - Must be acyclic - All nodes must define executor - All edges represent dependency ordering

3. EVENT STORE MODEL

Append-only log structure: Event: - id - timestamp - type - payload - previous_hash - hash
Guarantees: - immutability - replayability - auditability

4. MYTHOS POLICY DSL (DRAFT)

```
policy { rule "no_unapproved_execution" { when: pre_execution if: risk > threshold then: reject } rule  
"audit_all_actions" { when: any then: log_event } }
```

5. AGENT OS RUNTIME MODEL

Capabilities: - bounded permissions per agent - sandboxed execution - deterministic task execution
- event emission per action Execution Flow: DAG Node → Capability Check → Sandbox Execute →
Emit Event

6. SWARM DEPLOYMENT MODEL (K8S STYLE)

Nodes deployed as containers: agent-node: replicas: N resources: cpu: limited memory: bounded
Scheduler: - DAG partitioning - load balancing - fault isolation

7. SIMULATION ENGINE

Purpose: - evaluate DAG outcomes before execution Model: - branching futures - scoring function:
stability + safety + efficiency + success probability Output: - ranked execution paths

8. SYSTEM SAFETY GUARANTEES

- Mythos overrides all layers - No direct agent privilege escalation - No bypass of DAG compiler - Full traceability required - Deterministic replay mandatory

9. END-TO-END EXECUTION PIPELINE

Input → Memory → DAG Compiler → Simulation → Mythos Gate → Swarm Execution → Event Store → Evolution Loop